Processingをはじめよう

第9章

オブジェクト

目次

- クラスとオブジェクト
  - ブロックを作成
  - フィールドの追加
  - コンストラクタの作成とフィールドへの代入
  - メソッドの追加

オブジェクト

- オブジェクト＝もの
  - 関連しあう変数と関数の集合体
    - 変数 ⇒ フィールド、インスタンス変数
    - 関数 ⇒ メソッド
  - データと命令を結び付けたもの
  - 関連しあうデータと、それに働きかける命令や振る舞いを一つにまとめたもの

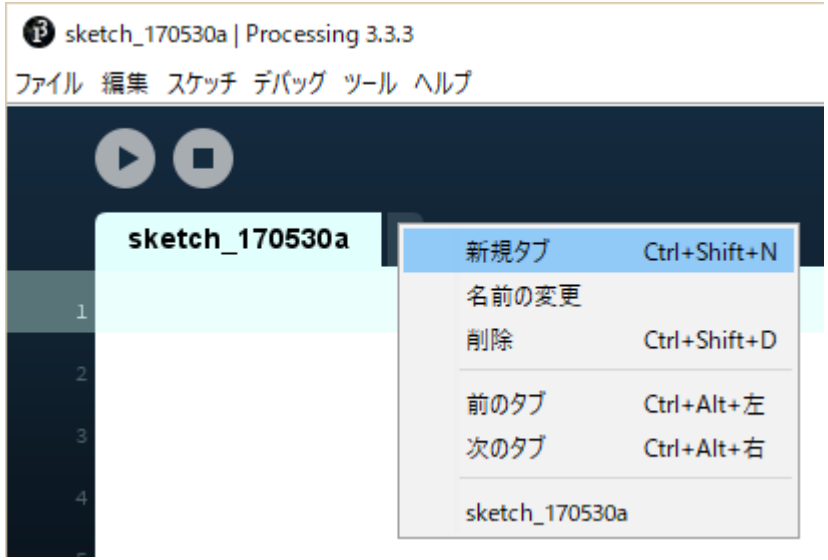
クラスとオブジェクト

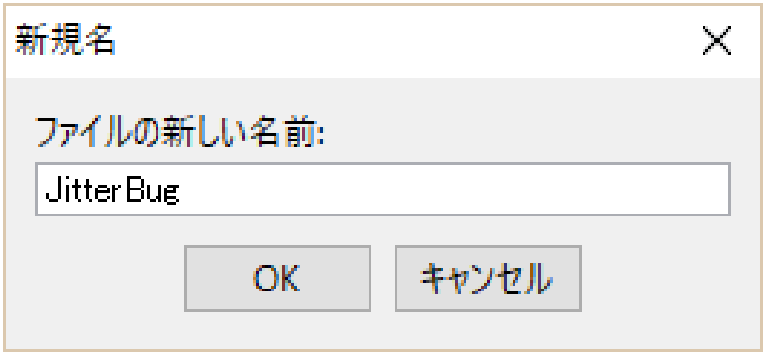
- クラス＝オブジェクトの仕様書
  - 建築に例えると・・・
    - クラス：家の設計図、オブジェクト：家そのもの
- クラスの定義

1. クラスのブロックを作成
2. フィールドを追加
3. コンストラクタの作成とフィールドへの代入
4. メソッドの追加

クラスの定義

- Example7-9 クラスを使って書き換え
- 新規タブを選択、クラスファイルを追加
  - クラス名 JitterBug





Example 7-9

```
float speed = 2.5;
int diameter = 20;
float x;
float y;

void setup() {
  size(240, 120);
  x = width/2;
  y = height/2;
}

void draw() {
  x += random(-speed, speed);
  y += random(-speed, speed);
  ellipse(x, y, diameter, diameter);
}
```

変更①　クラスのブロックを作成

```
float speed = 2.5;
int diameter = 20;
float x;
float y;

void setup() {
  size(240, 120);
  x = width/2;
  y = height/2;
}

void draw() {
  x += random(-speed, speed);
  y += random(-speed, speed);
  ellipse(x, y, diameter, diameter);
}
```

```
// 新規タブ JitterBug

class JitterBug {

}
```

クラスの
ブロック

```
class クラス名 {

}
```

クラス名は大文字で開始する
単語の頭文字を大文字にする

変更②　フィールドの追加

```
float speed = 2.5;
int diameter = 20;
float x;
float y;

void setup() {
  size(240, 120);
  x = width/2;
  y = height/2;
}

void draw() {
  x += random(-speed, speed);
  y += random(-speed, speed);
  ellipse(x, y, diameter, diameter);
}
```

```
// 新規タブ JitterBug

class JitterBug {
  float x;
  float y;
  int diameter;
  float speed = 0.5;
}
```

コンストラクタで
値を代入するフィールドは
フィールドの宣言のみ行う

それ以外のフィールドは
フィールドの宣言と
初期化を行う

```
class クラス名 {
  型 フィールド名;
}
```

変更③　コンストラクタの作成

```
void setup() {
  size(240, 120);
  x = width/2;
  y = height/2;
}

void draw() {
  x += random(-speed, speed);
  y += random(-speed, speed);
  ellipse(x, y, diameter, diameter);
}
```

```
// 新規タブ JitterBug

class JitterBug {
  float x;
  float y;
  int diameter;
  float speed = 0.5;

  JitterBug(){}
}
```

コンストラクタは
クラス名と同じ名前の
特別なメソッド

```
class クラス名 {
  型 フィールド名;

  クラス名(){}
}
```

変更③　フィールドへの代入

```
void setup() {
  size(240, 120);
  x = width/2;
  y = height/2;
}
```

```
class クラス名 {
  型 フィールド名;

  クラス名(引数型 引数名){
    フィールド名 = 引数名;
  }
}
```

```
// 新規タブ JitterBug

class JitterBug {
  float x;
  float y;
  int diameter;
  float speed = 0.5;

  JitterBug( float tempX, float tempY,
             float tempDiameter ) {
    x = tempX;
    y = tempY;
    diameter = tempDiameter;
  }

}
```

同じ型

コンストラクタで
値を代入するフィールド

値を代入するための引数

各フィールドに引数を代入する

変更④　メソッドの追加

```
class クラス名 {
  型 フィールド名;

  クラス名(引数型 引数名){
    フィールド名 = 引数名;
  }

  戻り値の型 フィールド名(引数型 引数名){
    文;
    return 式;
  }
}
```

```
void draw() {
  x += random(-speed, speed);
  y += random(-speed, speed);
  ellipse(x, y, diameter, diameter);
}
```

draw()内の各処理をメソッドへ

```
// 新規タブ JitterBug

class JitterBug {
  float x;
  float y;
  int diameter;
  float speed = 0.5;

  JitterBug( float tempX, float tempY,
                  float tempDiameter ) {
    x = tempX;
    y = tempY;
    diameter = tempDiameter;
  }

  void move(){}

  void display(){}

}
```

JitterBugが移動するメソッド move()追加

JitterBugを表示するメソッド display()追加

変更④　メソッドの追加

```
void setup() {
  size(240, 120);
  x = width/2;
  y = height/2;
}

void draw() {
  x += random(-speed, speed);
  y += random(-speed, speed);
  ellipse(x, y, diameter, diameter);
}
```

Example 9-1 につづく

```
class JitterBug {
  float x;
  float y;
  int diameter;
  float speed = 0.5;

  JitterBug( float tempX, float tempY,
                 float tempDiameter ) {
    x = tempX;
    y = tempY;
    diameter = tempDiameter;
  }

  void move(){
    x += random(-speed, speed);
    y += random(-speed, speed);
  }

  void display(){
    ellipse(x, y, diameter, diameter);
  }

}
```

JitterBugが移動するメソッド
move()追加

JitterBugを表示するメソッド
display()追加

オブジェクトを作る

- Example 9-1 (EX_10_01)
  - クラスをもとにオブジェクトを作る
    - オブジェクト変数を宣言
    - new演算子を使ってオブジェクトを生成（初期化）
  - メソッドの実行
    - オブジェクト変数名.メソッド(引数);

Example 9-1

```
JitterBug bug;

void setup() {
  size(240, 120);
  bug = new JitterBug(width/2,
                      height/2, 20);
}

void draw() {
  bug.move();
  bug.display();
}
```

オブジェクト変数の宣言
クラス名 オブジェクト変数名;

new演算子を使って
オブジェクトを生成（初期化）

オブジェクト変数名 =
new コンストラクタ(引数);

メソッドの実行
オブジェクト変数名.メソッド(引数);

変更なし

```
class JitterBug {
  float x;
  float y;
  int diameter;
  float speed = 0.5;

  JitterBug( float tempX, float tempY,
                 float tempDiameter ) {
    x = tempX;
    y = tempY;
    diameter = tempDiameter;
  }

  void move(){
    x += random(-speed, speed);
    y += random(-speed, speed);
  }

  void display(){
    ellipse(x, y, diameter, diameter);
  }

}
```

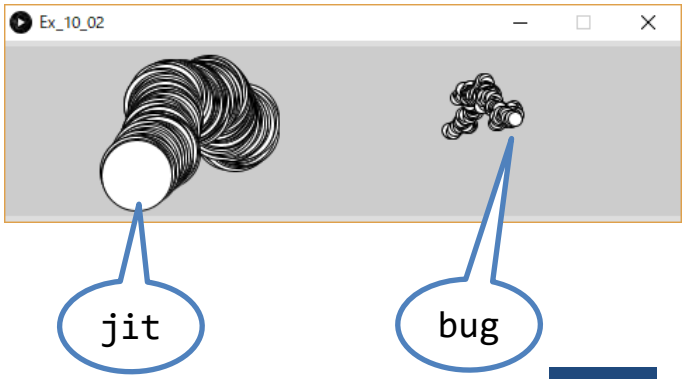
複数のオブジェクトを作る

- Example 9-2 (EX_10_02)
  - 同じクラスから2つのオブジェクトを作成

```
JitterBug jit;
JitterBug bug;
```

  - フィールドやメソッドにアクセスするドット(.)演算子

```
jit.move();      //jitが移動する
jit.display();   //jitを表示する
bug.move();      //bugが移動する
bug.display();   //bugを表示する
```



Example 9-2

```
JitterBug jit;
JitterBug bug;
```

オブジェクト変数
jit, bugの宣言

```
void setup() {
  size(240, 120);
  jit = new JitterBug(width*0.33,
                      height/2, 50);
  bug = new JitterBug(width*0.66,
                      height/2, 10);
}
```

jit, bugを
コンストラクタで初期化

```
void draw() {
  jit.move();
  jit.display();
  bug.move();
  bug.display();
}
```

jitが移動する
jitを表示する

bugが移動する
bugを表示する

変更なし

```
class JitterBug {
  float x;
  float y;
  int diameter;
  float speed = 0.5;

  JitterBug( float tempX, float tempY,
                 float tempDiameter ) {
    x = tempX;
    y = tempY;
    diameter = tempDiameter;
  }

  void move(){
    x += random(-speed, speed);
    y += random(-speed, speed);
  }

  void display(){
    ellipse(x, y, diameter, diameter);
  }

}
```